



Setup e arquitetura

Desenvolvimento Móvel

Aula 28/36

Flutter

Widget

Provider

Riverpod

Api

Http

Firebase

Firestore

Conceito

Setup do projeto final: flutter create com estrutura de pastas organizada. Arquitetura recomendada: lib/data (models, repositories, datasources), lib/domain (entities, usecases, repository interfaces), lib/presentation (pages, widgets, controllers/providers), lib/core (constants, errors, network, utils). Dependencias principais no pubspec.yaml: riverpod/provider (estado), dio/http (API), firebase_core + firebase_auth + cloud_firestore (Firebase), go_router (navegacao), json_serializable + build_runner (serializacao). Git: repositorio no GitHub com branch main e dev, commits semanticos (feat:, fix:, chore:, docs:). Flavors configuram ambientes dev/prod com diferentes app names, icones e URLs de API usando --flavor no flutter build.

lib/

data

models / repositories

domain

entities / usecases

presentation

pages / widgets / providers

core

constants / utils / errors

Git + flavors + pubspec.yaml



Atividade Prática

Crie o repositório GitHub para o projeto final (um por grupo). Execute flutter create com o nome do app. Organize a estrutura de pastas conforme a arquitetura escolhida. Adicione as dependências principais no pubspec.yaml. Faça o primeiro commit com mensagem semântica (chore: initial project setup) e push para main. Crie branch dev para desenvolvimento.



Pesquisa

Pesquise sobre flavors no Flutter: como configurar diferentes ambientes (dev/staging/prod) usando --flavor, com diferentes nomes de app, ícones, bundle ID e URLs de API. Como ler variáveis de ambiente com --dart-define ou pacote flutter_config?



Requisitos

Flutter SDK. Git instalado. Conta no GitHub.